

JavaScript Day 2

What Is an Execution Context?

An Execution Context is like a box or environment where JavaScript code is **evaluated and executed**.

Think of it like:

A theater stage where a specific scene (function or block of code) is being played. Each time a new scene begins, a new stage setup is created.

Types of Execution Contexts:

1. Global Execution Context (GEC)

- Created when the JavaScript file first runs.
- There is only **one** global context in a program.
- The window object (in browsers) or global object (in Node.js) is part of it.
- The `this` keyword in global context refers to the global object (window in browsers).

2. Function Execution Context

- Created every time a function is called.
- Each has its own variables, scope, and value of `this`.

3. Eval Execution Context (*rarely used*)

- Created when code is run inside `eval()`, which is discouraged.
-

Inside an Execution Context:

Each execution context has two main phases:

1. Creation Phase

- Creates the **Variable Object (VO)**: stores variables, function declarations.
- Sets the **scope chain**.
- Sets the value of **this**.

2. Execution Phase

- Executes the code line by line.
- Assigns values to variables.
- Runs function code.

What Is the Call Stack?

The **Call Stack** is a **stack data structure** (LIFO – Last In, First Out) used to keep track of function calls.

Imagine a stack of plates. You always place the new plate on top, and you remove the top plate first.

How It Works:

1. When JavaScript runs, the **Global Execution Context** is created and **pushed** onto the stack.

2. When a function is called, a new **Function Execution Context** is created and **pushed** on top.
 3. When that function finishes, its context is **popped** off the stack.
 4. This continues until the stack is empty (program finishes).
-

Example:

js:

```
function greet() {  
    console.log("Hello!");  
}  
  
function start() {  
    greet();  
    console.log("Starting...");  
}  
  
start();
```

Call Stack Trace:

1. **Global Context** → pushed to stack.
2. **start()** called → start context pushed.
3. Inside start, **greet()** is called → greet context pushed.
4. **greet()** finishes → context popped.
5. **start()** prints "Starting..." → finishes → context popped.
6. Stack is now back to **Global** context.

Visual Representation:

```
| greet()          | ← top (last in)
| start()          |
| Global Context   |
```

After greet() finishes:

```
| start()          |
| Global Context   |
```

Stack Overflow (Common Error)

If functions keep calling each other endlessly:

js

```
function recurse() {
  recurse();
}
recurse(); // Maximum call stack size exceeded
```

JavaScript runs out of space in the stack, and throws a **stack overflow** error.

-
-

What Is Hoisting?

Hoisting is JavaScript's behavior of **moving declarations to the top** of the current **scope** (either global or function) **before the code is executed**.

Key idea:

Declarations are hoisted, but initializations are not.

This means that variables and functions are “**seen**” by JavaScript before they are **defined** in the code.

Variable Hoisting

Let's see an example:

```
js
console.log(x); // undefined
var x = 5;
console.log(x); // 5
```

What actually happens:

Behind the scenes, JavaScript does this:

```
js
var x;           // declaration is hoisted
console.log(x);  // undefined
x = 5;           // initialization stays in place
console.log(x);  // 5
```

So the **declaration** `var x` is hoisted to the top, but the **assignment** `(= 5)` is not.

Common Pitfall

```
js
console.log(y); // ReferenceError
let y = 10;
```

Why? Because **let** and **const** are **NOT** hoisted in the same way as **var**.

They are hoisted, but they stay in a “**temporal dead zone**” (TDZ) from the start of the block until the line they’re declared. You can’t access them before the declaration. Time between entering a block and the actual declaration of **let** or **const** is the temporal dead zone. Classes are hoisted, **but not initialized** (just like **let** and **const**). They are in the TDZ.

Hoisting with var, let, and const

Declaration	Hoisted?	Initialized to undefined ?	Usable before declared?
var	✓ Yes	✓ Yes	✓ Yes (but undefined)
let	✓ Yes	✗ No	✗ No (TDZ error)
const	✓ Yes	✗ No	✗ No (TDZ error)

Function Hoisting

Function Declaration

js

```
greet(); // Hello!
```

```
function greet() {  
  console.log("Hello!");  
}
```

✔ Works because function **declarations** are **fully hoisted**, including the body.

Function Expression

```
js
sayHi(); // ✗ TypeError: sayHi is not a function

var sayHi = function () {
  console.log("Hi!");
};
```

✖ This doesn't work because sayHi is hoisted **as a variable** (var), but not the function body.

Behind the scenes:

```
js
var sayHi;
sayHi(); // TypeError
sayHi = function () {
  console.log("Hi!");
};
```

Date

-

Task	Use
Create date	<code>new Date()</code> or <code>new Date('2025-06-25')</code>
Get parts	<code>getFullYear()</code> , <code>getMonth()</code> , etc.
Set parts	<code>setFullYear()</code> , <code>setMonth()</code> , etc.
Compare dates	Use <code><</code> , <code>></code> , or subtract dates

Format	<code>toDateString(), toLocaleDateString(),</code>
date	<code>Intl.DateTimeFormat</code>

Absolutely! Understanding **scope** in JavaScript is *essential* because it determines **where variables can be accessed or modified** in your code.

Let's walk through this step by step — with points, subpoints, and examples — so you truly understand how **scope** works.

What is Scope in JavaScript?

Scope is the current **context of execution** — it defines **accessibility** (visibility) of variables.

You can think of scope like **bubbles**: a variable can only be seen inside the bubble where it's created and possibly in inner bubbles.

Types of Scope in JavaScript

◆ 1. Global Scope

- Variables declared **outside of all functions or blocks**.
- Can be accessed **anywhere** in your code.

JavaScript

```
var name = "Alice";

function greet() {
  console.log(name); //  Accessible
}
```



```
greet();
```

♦ 2. Function Scope

- Variables declared **inside a function** using `var`, `let`, or `const` are only accessible **within that function**.

JavaScript

```
function sayHello() {  
  let message = "Hello!";  
  console.log(message); // ✅ Accessible  
}  
  
console.log(message); // ❌ ReferenceError
```

♦ 3. Block Scope

- Variables declared inside `{ }` (e.g., in `if`, `for`, `while`, or just plain `{ }`) using `let` or `const` are **block-scoped**.

JavaScript

```
{  
  let color = "blue";  
  const age = 25;  
  console.log(color); // ✅  
}  
  
console.log(color); // ❌ ReferenceError
```

var does not have block scope, only function scope!

◆ 4. Lexical Scope (Static Scope)

- JavaScript uses **lexical scoping**, meaning **scope is determined at the time of writing code**, not during runtime.

JavaScript

```
function outer() {  
  let outerVar = "I'm outside!";  
  
  function inner() {  
    console.log(outerVar); // ✅ Can access parent  
    scope  
  }  
  
  inner();  
}  
outer();
```

Scope Chain

When a variable is accessed:

1. JavaScript **looks in the current scope**.
2. If not found, it goes **one level up**, and keeps going up.
3. If not found in the global scope, you get a **ReferenceError**.

Example:

JavaScript

```
let a = 1;

function outer() {
  let b = 2;

  function inner() {
    let c = 3;
    console.log(a, b, c); // ✓ All accessible
  }

  inner();
}
outer();
```

✗ Shadowing

A variable declared in a **local scope** can "**shadow**" (**hide**) a variable of the same name in an outer scope.

JavaScript

```
let x = 10;

function test() {
  let x = 20; // Shadows global x
  console.log(x); // 20
}

test();
console.log(x); // 10
```

var vs let/const in Scope

Keyword	Scope Type	Block Scoped?	Function Scoped?
var	Function Scope	✗ No	✓ Yes
let	Block Scope	✓ Yes	✓ Yes
const	Block Scope	✓ Yes	✓ Yes

Example Combining It All

JavaScript

```
let globalVar = "Global";

function outer() {
  let outerVar = "Outer";

  if (true) {
    let blockVar = "Block";
    console.log(globalVar); // ✓
    console.log(outerVar); // ✓
    console.log(blockVar); // ✓
  }

  // console.log(blockVar); // ✗ ReferenceError
}

outer();
```

Summary

Scope Type	Where Defined	Accessibility
Global Scope	Outside all blocks	Everywhere
Function Scope	Inside a function	Only inside that function
Block Scope	Inside <code>{ }</code> using <code>let/const</code>	Only within that block
Lexical Scope	Nested function scope	Can access parent scope variables

Would you like to practice this with real code snippets, or move on to **closures**, which build on the idea of scope?

Closure:

Inner function remembers the variable addresses of outer functions even after they are finished. Closure remembers the variable addresses not the values itself. For example

✅ Step 1: The Code

JavaScript

```
function makeFuncs() {  
  let funcs = [];  
  
  for (var i = 0; i < 3; i++) {  
    funcs.push(function () {  
      console.log(i);  
    });  
  }  
  
  return funcs;  
}
```

```
}
```

```
const f = makeFuncs(); // Step A  
f[0](); // Step B  
f[1](); // Step C  
f[2](); // Step D
```

Before We Start: Core Concepts

♦ What's a Closure?

A function that **remembers its outer scope**, even after that scope has "finished."

♦ Execution Stack vs Callback Queue

- **Call Stack** (or Execution Stack): Sync operations (functions called immediately).
- **Callback Queue**: Async operations (like `setTimeout`, events). Not involved here.

 In this case, **everything is synchronous**, so **only the Call Stack is involved**.

Step-by-Step Dry Run

 **Step A:** `const f = makeFuncs();`

✅ **Execution Stack:**

- `Global()`

- calls → `makeFuncs()`
-

Inside `makeFuncs()`

- Declares `let funcs = []`
- Starts a `for` loop: `var i = 0; i < 3; i++`

Loop iteration breakdown:

🔥 Remember: `var i` is function-scoped, so **one `i` is shared** across all 3 iterations!

🌀 Loop #1:

- `i = 0`
- Pushes a function: `() => console.log(i)`
- That function **remembers the `i` variable**, not the value `0`.

🌀 Loop #2:

- `i = 1`
- Pushes another function that **remembers the same `i` variable**.

🌀 Loop #3:

- `i = 2`
- Pushes third function → still **closing over the same `i`**

Loop ends: `i = 3`

✓ makeFuncs() Returns:

- Returns the `funcs` array containing 3 functions:

JavaScript

```
[  
  function () { console.log(i); },  
  function () { console.log(i); },  
  function () { console.log(i); }  
]
```

All 3 functions **share a closure over the same `i` variable**, which is now `3`.

Step B: `f[0]()`;

✓ Call Stack:

None

```
Global()  
→ f[0]()
```

- - Calls the first function: `console.log(i)`
 - It looks for `i` in its closure → finds `i = 3`
 - ✓ Prints: `3`
-

Step C: `f[1]()`;

Same as above:

- Function still **shares closure** with same `i`
 -  Prints: `3`
-

Step D: `f[2]()`;

Again:

- Closure over the same `i`
 -  Prints: `3`
-

WHY All Print 3?

Because:

- `var` is **function-scoped**, not block-scoped.
 - All 3 functions **close over the same** `i` — not a copy.
 - By the time `f[0]()`, `f[1]()`, etc. are called, `i` has become `3`.
-

How to Fix It: Use `let`

JavaScript

```
function makeFuncs() {  
  let funcs = [];  
  
  for (let i = 0; i < 3; i++) {  
    funcs.push(function () {  
      console.log(i);  
    });  
  }  
  
  return funcs;  
}
```

Why This Works:

- **let** is **block-scoped**
- So each loop iteration has a **new, separate i**
- Each function gets a **new closure** with its own **i**

✓ Output:

None

0
1
2



Summary

Concept

What Happens

<code>var i</code>	Shared across all loop iterations
Closure	Each function remembers the same i
After loop ends	<code>i = 3</code> → all functions print 3
Execution model	All code is synchronous → Call Stack only
Fix	Use <code>let</code> for block-scoped <code>i</code> per iteration

With var:

Iteration	i Value	Closure Created	Captures
0	0	Function A	same <code>i</code>
1	1	Function B	same <code>i</code>
2	2	Function C	same <code>i</code>
After loop <code>i = 3</code>			all see 3

With let

Iteration	Closure Created	Captures
0	Function A	own <i>i</i>
1	Function B	own <i>i</i>
2	Function C	own <i>i</i>

Object methods:

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}
//Now we create an object of this function/method
const myCar = new Car("Eagle", "Talon TSi", 1993);
```

Using object.create():

```
// Animal properties and method encapsulation
const Animal = {
  type: "Invertebrates", // Default value of properties
  displayType() {
    // Method which will display type of Animal
    console.log(this.type);
  },
};
```

```
// Create new animal type called animal1
const animal1 = Object.create(Animal);
animal1.displayType(); // Logs: Invertebrates
```

```
// Create new animal type called fish
const fish = Object.create(Animal);
fish.type = "Fishes";
fish.displayType(); // Logs: Fishes
```

Object methods:

```
const Manager = {
  name: "Karina",
  age: 27,
  job: "Software Engineer",
};
const Intern = {
  name: "Tyrone",
  age: 21,
  job: "Software Engineer Intern",
};
```

```
function sayHi() {
  console.log(`Hello, my name is ${this.name}`);
}
```

```
// add sayHi function to both objects
Manager.sayHi = sayHi;
Intern.sayHi = sayHi;
```

```
Manager.sayHi(); // Hello, my name is Karina
Intern.sayHi(); // Hello, my name is Tyrone
```

Object getters & setters:

```
const myObj = {
  a: 7,
  get() {
    return this.a + 1;
  },
  set (x) {
    this.a = x / 2;
  }
};
```

```

    },
  };

  console.log(myObj.a); // 7
  console.log(myObj.get()); // 8, returned from the get b() method
  myObj.set(50); // Calls the set c(x) method
  console.log(myObj.a); // 25

```

Comparing objects:

// Two variables, two distinct objects with the same properties

```

const fruit = { name: "apple" };
const anotherFruit = { name: "apple" };

```

```

fruit == anotherFruit; // return false
fruit === anotherFruit; // return false

```

Another

// Two variables, a single object

```

const fruit = { name: "apple" };
const anotherFruit = fruit; // Assign fruit object reference to anotherFruit

```

// Here fruit and anotherFruit are pointing to same object

```

fruit == anotherFruit; // return true
fruit === anotherFruit; // return true

```

```

fruit.name = "grape";
console.log(anotherFruit); // { name: "grape" }; not { name: "apple" }

```

Object Methods:

There are different methods to manipulate an object. Let us see some of the available methods.

Object.assign: To copy an object without modifying the original object
JS:

```

const person = {
  firstName: 'Asabeneh',
  age: 250,

```

```

country: 'Finland',
city: 'Helsinki',
skills: ['HTML', 'CSS', 'JS'],
title: 'teacher',
address: {
  street: 'Heitamienkatu 16',
  pobox: 2002,
  city: 'Helsinki'
},
getPersonInfo: function() {
  return `I am ${this.firstName} and I live in ${this.city}, ${this.country}. I am
  ${this.age}.`
}
}

```

```

//Object methods: Object.assign, Object.keys, Object.values, Object.entries
//hasOwnProperty

```

```

const copyPerson = Object.assign({}, person)
console.log(copyPerson)

```

Getting object keys using Object.keys()

Object.keys: To get the keys or properties of an object as an array

```

const keys = Object.keys(copyPerson)
console.log(keys) //['firstName', 'age', 'country', 'city', 'skills', 'title', 'address',
'getPersonInfo']
const address = Object.keys(copyPerson.address)
console.log(address) //['street', 'pobox', 'city']

```

Getting object values using Object.values()

Object.values: To get values of an object as an array

```

const values = Object.values(copyPerson)
console.log(values)

```

Getting object keys and values using Object.entries()

Object.entries: To get the keys and values in an array

```
const entries = Object.entries(copyPerson)
console.log(entries)
```

Checking properties using hasOwnProperty()

hasOwnProperty: To check if a specific key or property exist in an object

```
console.log(copyPerson.hasOwnProperty('name'))
console.log(copyPerson.hasOwnProperty('score'))
```

Sets and maps from:

https://github.com/Asabeneh/30-Days-Of-JavaScript/blob/master/10_Day_Sets_and_Maps/10_day_Sets_and_Maps.md